Enpal Senior Analytics Engineer Assessment

Setup

1. Download Docker Desktop (if you don·t have installed) using the official website, install and launch.
2. Fork this Github project to you Github account. Clone the forked repo to your device.
3. Open your Command Prompt or Terminal, navigate to that folder, and run the command `docker compose up`.
4. Now you have launched a local Postgres database with the following credentials:

```
Host: localhost
User: admin
Password: admin
Port: 5432
```

5. Connect to the db via a preferred tool (e.g. DataGrip, Dbeaver etc)
6. Install dbt-core and dbt-postgres using pip (if you don·t have) on your preferred environment.
7. Now you can run `dbt run` with the test model and check public_pipedrive_analytics schema to see the dbt result (with one test model)

Project

1. Remove the test model once you make sure it works
2. Dive deep into the Pipedrive CRM source data to gain a thorough understanding of all its details. (You may also research the Pipedrive CRM tool terms).
3. Define DBT sources and build the necessary layers organizing the data flow for optimal relevance and maintainability.
4. Build a reporting model (rep_sales_funnel_monthly) with monthly intervals, incorporating the following funnel steps (KPIs):
   Step 1: Lead Generation
   Step 2: Qualified Lead
   Step 2.1: Sales Call 1
   Step 3: Needs Assessment
   Step 3.1: Sales Call 2
   Step 4: Proposal/Quote Preparation
   Step 5: Negotiation
   Step 6: Closing
   Step 7: Implementation/Onboarding
   Step 8: Follow-up/Customer Success
   Step 9: Renewal/Expansion
5. Column names of the reporting model: `month`, `kpi_name`, `funnel_step`, `deals_count`
6. ·Git commit· all the changes and create a PR to your forked repo (not the original one). Send your repo link to us.

Solution Documentation

1. Project Overview

This project implements a robust dbt pipeline to transform raw Pipedrive CRM data into a monthly Sales Funnel Report. The solution focuses on data quality, performance optimization, and maintainability, ensuring accurate tracking of deals as they progress through 11 distinct funnel steps.

2. Architecture & Models

The project follows a layered architecture:

· Staging Layer (`models/staging`)

- **Purpose**: Clean, standardize, and type-cast raw data.
- **Key Models**:
  - `stg_stages`: Standardizes stage names (e.g., fixing capitalization inconsistencies like "Qualified lead" -> "Qualified Lead") using `initcap()`.
  - `stg_deal_changes`: Extracts deal history and parses timestamps.
  - `stg_activity`: Filters and prepares activity data for sales calls.
  - `stg_activity_types`: Maps activity codes to human-readable names (e.g., "Sales Call 1").
  - `stg_users`: Cleans user/sales rep data.
  - `stg_fields` & `stg_lost_reasons`: Handles metadata for deal fields and lost reasons.

· Intermediate Layer (`models/intermediate`)

- **Purpose**: Business logic, complex joins, and aggregations.
- **Key Models**:
  - `int_deal_stage_changes`: **(Incremental Model)** Tracks the historical movement of deals through stages. It uses a `delete+insert` strategy to handle updates efficiently.
  - `int_deal_creation`: Isolates the "Lead Generation" event (Step 1).
  - `int_sales_call_1` & `int_sales_call_2`: Specifically isolates "Sales Call 1" (Step 2.1) and "Sales Call 2" (Step 3.1) events.

· Reporting Layer (`models/reporting`)

- **Purpose**: Final presentation-ready tables.
- **Key Model**: `rep_sales_funnel_monthly`
  - **Materialization**: `Table` (ensures 100% consistency for historical reporting).
  - **Logic**: Unions data from deal creation, stage changes, and sales calls into a unified monthly view.
  - **Performance**: Uses post-hooks to create database indexes on `month` and `kpi_name`.

3. Key Features & Engineering Highlights

· Incremental Loading Strategy

The `int_deal_stage_changes` model is configured as **incremental** to optimize performance as data grows.

- **Strategy**: `delete+insert`
- **Unique Key**: `change_id` (Surrogate key generated from `deal_id` + `stage_id` + `timestamp`)
- **Benefit**: Only processes new or changed records during runs, significantly reducing compute time for large datasets.

· Custom Macros

- `filter_increasing_stage_dates`: A sophisticated macro that cleans dirty data where stage timestamps might be out of order. It ensures that a deal cannot enter Stage 3 *before* Stage 2.
- `generate_index`: Automates the creation of Postgres indexes on downstream reporting tables to speed up BI queries.

· Data Quality Testing

- **38+ Tests Implemented**: Covering `unique`, `not_null`, and `accepted_values`.
- **Custom Generic Test**: `stage_dates_increase` validates that for every deal, the `stage_entered_at` timestamp strictly increases as the `stage_id` increases.
- **Fixes**: Addressed raw data issues such as inconsistent capitalization in stage names and duplicate activity IDs.

4. Interpretations & Insights

The final report `rep_sales_funnel_monthly` allows stakeholders to answer questions like:

- **Volume**: "How many new leads were generated in October?" (Filter: `Step 1: Lead Generation`)
- **Conversion**: "How many deals moved from 'Qualified Lead' to 'Needs Assessment'?"
- **Velocity**: By analyzing the `month` column, we can see seasonal trends in deal flow.
- **Sales Activity**: Tracks specific touchpoints like "Sales Call 1" and "Sales Call 2" as distinct funnel steps, highlighting the effort required to move deals forward.

5. How to Run

1. **Start Database**:

   ```
   docker compose up -d
   ```

2. **Check Connection to Postgres DB & Install Dependencies**:

   ```
   dbt debug
   dbt deps
   ```

3. **Run Models**:

   ```
   dbt run
   ```

4. **Run Tests**:

   ```
   dbt test
   ```

5. **Generate Documentation**:

   ```
   dbt docs generate
   dbt docs serve
   ```